

ULTRON 2.7 Phase 1 Execution Report

Basic text-first prototype for a safe Jarvis-style laptop assistant

Project	ULTRON 2.7
Phase	Phase 1: Core Text Prototype
Workspace	Local workspace checkout: U2.7
Status	Implemented and verified

Executive Summary

Phase 1 has been executed as a safe text-first assistant prototype. The implementation converts a user command into a structured tool call, validates the call against an allowlisted schema, applies risk and confirmation policy, runs only through a controlled executor, and writes an audit record.

The prototype intentionally defaults to dry-run execution. High-risk commands, such as deleting files, stop at a confirmation gate instead of executing directly.

Phase 1 Scope

- Build a command-line assistant entry point.
- Use the supplied dataset for exact and fuzzy command planning.
- Add deterministic regex fallbacks for common numeric commands.
- Define typed tool schemas and argument validation.
- Add policy gates for low, medium, high, and unsupported actions.
- Add a dry-run executor and audit log.
- Add tests and dataset evaluation for the basic prototype.

Implemented Architecture

The current Phase 1 pipeline is:

```
text command -> planner -> tool call -> validator -> policy -> executor -> audit log
```

The design follows the research paper’s recommendation that the LLM or planner should propose actions, while the application remains responsible for validation, permission checks, execution, and logging.

What Was Implemented

Component	File(s)	Purpose
CLI	src/ultron27/cli.py	Accepts a typed command, prints JSON output, and writes audit records.
Planner	src/ultron27/planner.py	Uses dataset matching plus deterministic fallbacks to produce a tool call.
Tool schemas	src/ultron27/tools.py	Defines allowed tools, required arguments, numeric ranges, and validation rules.
Policy	src/ultron27/policy.py	Allows safe commands, confirms risky commands, and blocks unsupported requests.
Executor	src/ultron27/executor.py	Runs dry-run actions by default and contains safe handlers for early tools.
Evaluation	src/ultron27/evaluate.py, scripts/evaluate_dataset.py	Measures planner quality against the supplied dataset splits.
Tests	tests/test_pipeline.py	Covers parsing, validation, policy, and dry-run execution behavior.
Docs	README.md, docs/architecture.md, docs/evaluation.md	Explains usage, architecture, evaluation, and next steps.

Safety Model

Phase 1 enforces a tool-based safety model instead of direct shell access.

Risk	Examples	Prototype behavior
None	clarification, unsupported request	No OS action is executed.
Low	open app, set volume, search files	Allowed after schema validation.
Medium	draft email, move file, calendar draft	Confirmation when configured by the tool.
High	delete file, send email, run script, shutdown	Confirmation required; destructive execution is not implemented.

Verification Results

The Phase 1 prototype was verified with unit tests, dataset evaluation, and a high-risk CLI safety check.

Check	Command	Result
Unit tests	python -m unittest discover -s tests	6 tests passed.
Dataset evaluation	python scripts/evaluate_dataset.py --split test	Intent 1.0, tool 1.0, argument exact match 0.9614, confirmation 1.0.

Check	Command	Result
High-risk command	<code>python -m ultron27 "delete project_report.txt"</code>	Policy returned <code>confirmation_required</code> .

The argument `exact-match` misses come from ambiguous synthetic dataset rows where similar or identical utterances have different generated file types, folders, or file names. That is documented in `docs/evaluation.md`.

Repository Map

- `src/ultron27`: assistant package and core Phase 1 logic.
- `scripts/evaluate_dataset.py`: dataset evaluation entry point.
- `tests/test_pipeline.py`: regression tests for command planning and safety gates.
- `data/jarvis_dataset_v2`: supplied synthetic command dataset.
- `docs/jarvis_research_paper.md`: supplied research paper.
- `docs/architecture.md`: local-first architecture summary.
- `docs/evaluation.md`: baseline metrics and dataset notes.

How To Run Phase 1

- 1 Set `PYTHONPATH` to `src` when running from the checkout, or install with `pip install -e .`
- 2 Run a safe command: `python -m ultron27 "set volume to 40 percent"`.
- 3 Check a high-risk gate: `python -m ultron27 "delete project_report.txt"`.
- 4 Run tests: `python -m unittest discover -s tests`.
- 5 Run evaluation: `python scripts/evaluate_dataset.py --split test`.

Recommended Next Phase

Phase 2 should focus on real low-risk laptop execution: app opening, file search, folder opening, note creation, reminders, clipboard, screenshot, volume, and brightness adapters. High-risk actions should remain preview-only until the confirmation UX is stronger.